

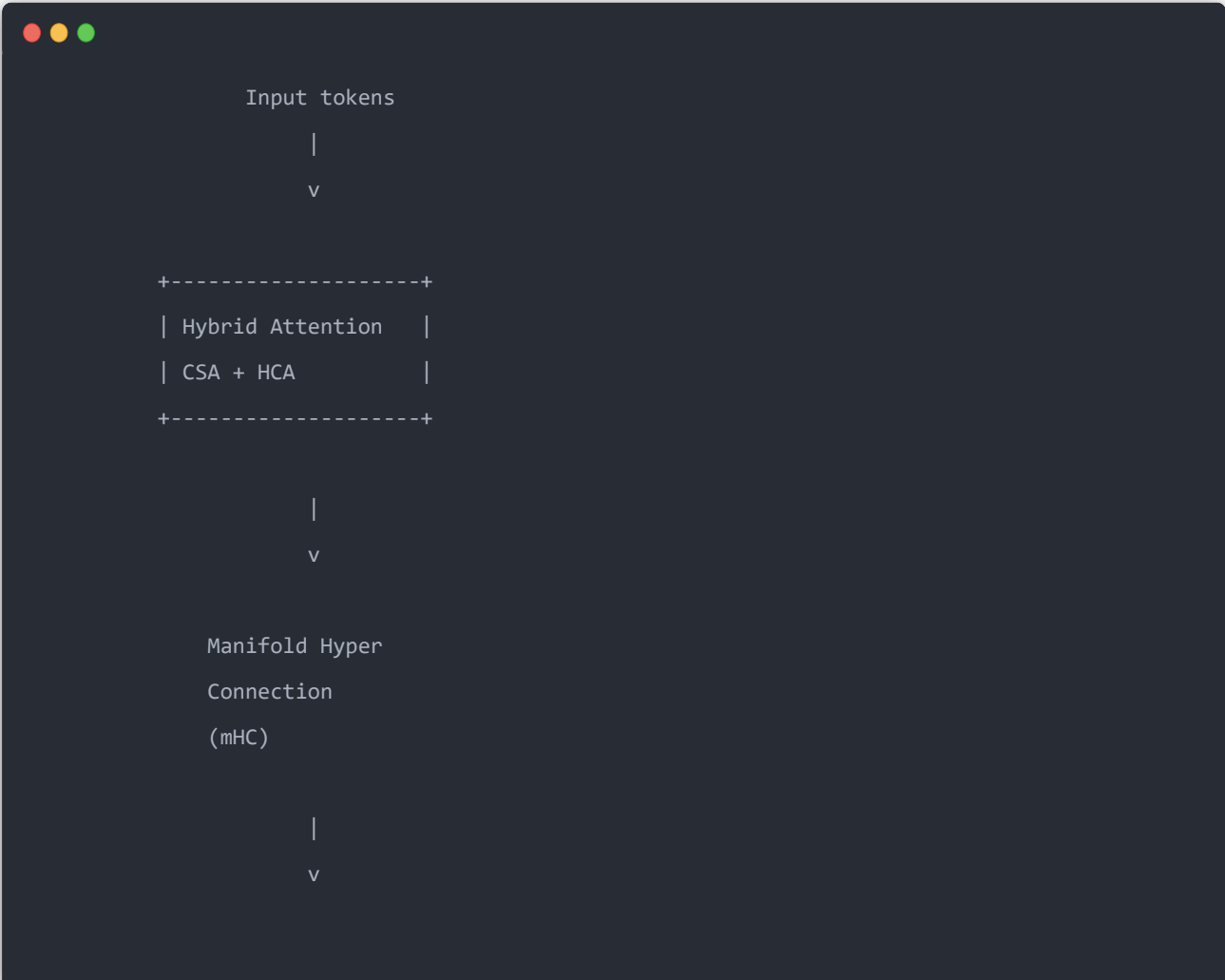
结合 **DeepSeek-V4 Technical Report** 和源码实现思路，重点从**模型结构、训练方法、为什么这样设计、和 DeepSeek-V3 的区别**几个角度分析。你之前重点关注 **SGLang 部署、B200 L2、FP4、MoE、KDA、AttnRes**，所以我会偏系统和推理视角讲。DeepSeek-V4 的核心目标是：

让 MoE 大模型具备百万 token 上下文能力，同时把 KV Cache 和推理 FLOPs 压下来。

官方报告定位就是 *Towards Highly Efficient Million-Token Context Intelligence*。V4-Pro 为 1.6T 参数 MoE、49B 激活；V4-Flash 为 284B 参数、13B 激活，两者支持百万 token context。

1. DeepSeek-V4 整体架构

先看整体：





相比 V3:

模块	DeepSeek-V3	DeepSeek-V4
Attention	MLA	CSA + HCA Hybrid Attention
Residual	普通 Add	mHC
MoE	DeepSeek MoE	升级版 MoE
Optimizer	AdamW	Muon
Quant	FP8	FP4 QAT
Context	128K级	1M token

2. 最大创新一：Hybrid Attention

这是 V4 的核心。

为什么？

因为：

Full Attention

复杂度：

$$O(n^2)$$

1M token:



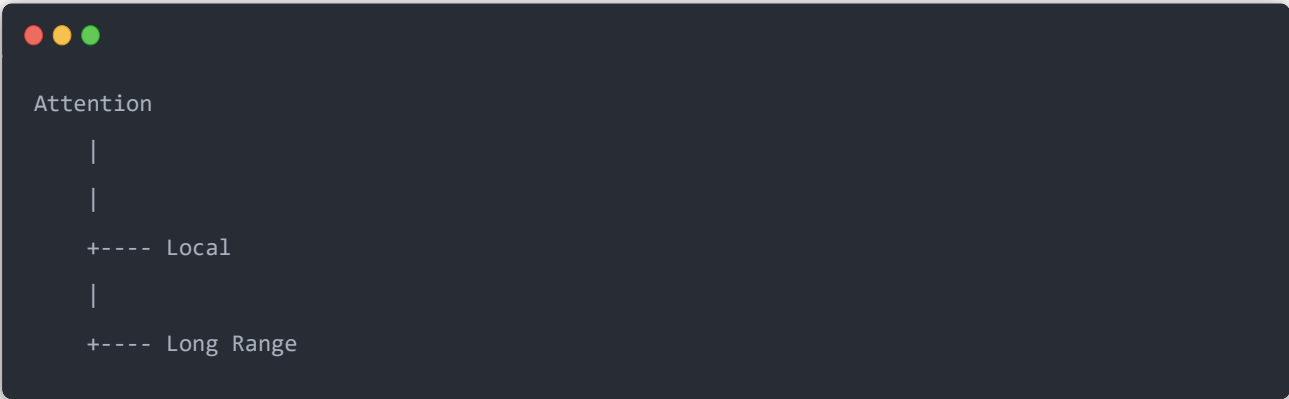
```
1,000,000 × 1,000,000
```

不可接受。

DeepSeek V3 用 MLA:

减少 KV cache。

V4 进一步:



```
Attention
```

```
|
```

```
|
```

```
+---- Local
```

```
|
```

```
+---- Long Range
```

拆成:

CSA

Compressed Sparse Attention

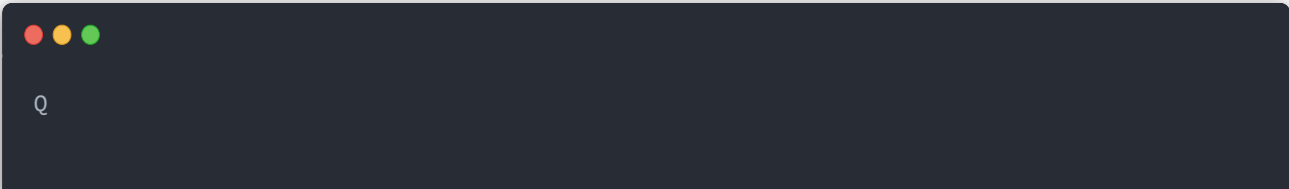
HCA

Heavily Compressed Attention

([GitHub](#))[3])

3. CSA 怎么工作?

传统 Attention:



```
Q
```

↓

所有 KV

↓

attention

CSA:

先压缩:



token:

1 2 3 4 5 6 7 8

compress

chunk1

chunk2

chunk3

生成:

compressed KV。

然后:

不是全部看:

而是:



query

|

indexer

|

选择 top-k chunks

|
attention

类似：



Google 搜索

不是扫描整个网页

而是先检索候选

数学：

原始：

$$Attention(Q, K, V)$$

CSA:

先：

$$K_c = f(K)$$

选择：

$$TopK(Q, K_c)$$

最后：

$$Attention(Q, K_{selected}, V_{selected})$$

所以：

KV cache 大幅下降。

4. HCA 是什么？

CSA:

稀疏。

但是:

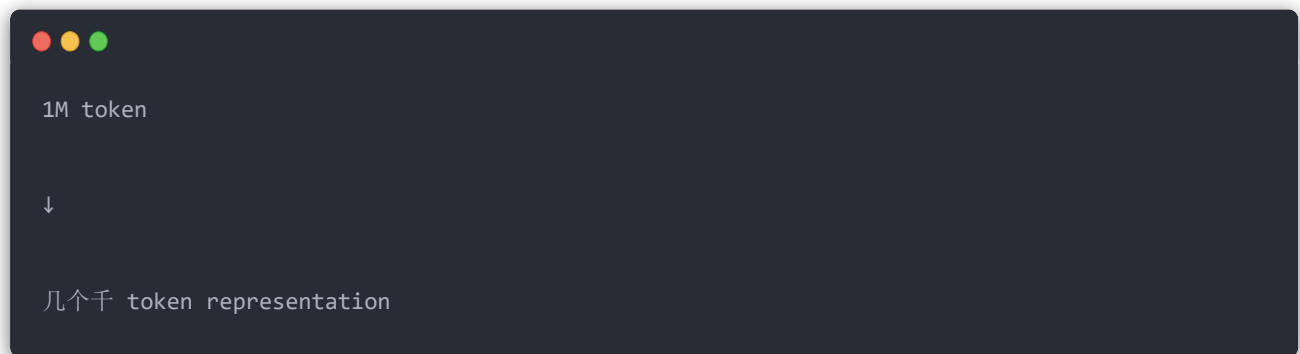
某些层需要全局信息。

所以:

加入:

Heavily Compressed Attention。

更激进压缩:



然后:

dense attention。

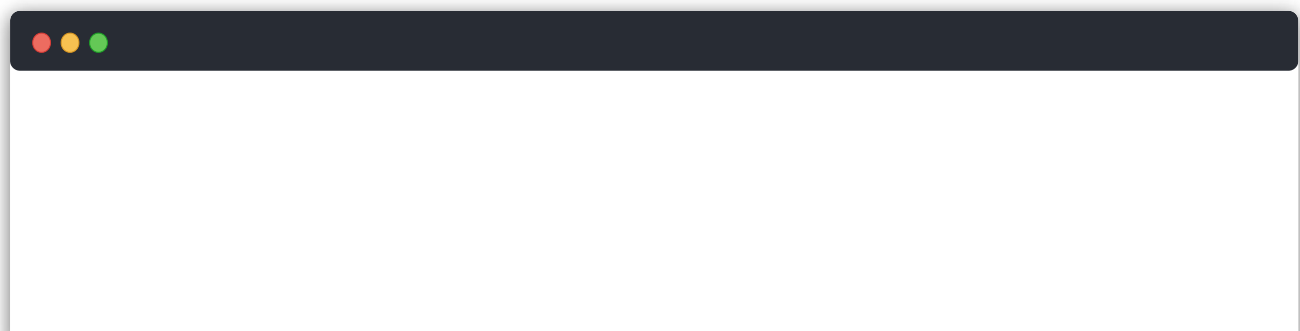
所以:

V4:

不是:

Linear Attention。

而是:



```
Local full attention
```

```
+
```

```
compressed global attention
```

这是非常重要区别。

5. 为什么不用 KDA / Delta Attention?

你之前问 Kimi K3 KDA。

对比：

KDA

思想：



维护状态

token流式更新

类似：

RNN memory。

DeepSeek V4:



仍然是 Attention

但是压缩KV

原因：

大模型 reasoning:

很多任务需要：

精确 retrieval。

例如：

代码：

```
第200000行定义变量

第800000行使用
```

纯 linear memory：

容易丢。

所以 V4 选择：

保留 Attention 能力，只减少 KV。

6. 创新二：mHC (Manifold-Constrained Hyper Connections)

这个对应你刚刚问的 Attention Residual。

但是方向不同。

Transformer residual：

$$x_{l+1} = x_l + F(x_l)$$

问题：

深层：

```
layer1
+
layer2
```



```
+  
...  
layer100
```

容易：

- signal amplification
 - representation collapse
-

mHC:

不直接：

$$x + F(x)$$

而是：

学习：

$$X_{l+1} ===== H_l X_l + F(X_l)$$

其中：

H:

不是任意矩阵。

约束：

双随机矩阵：

```
● ● ●  
  
每行sum=1  
每列sum=1
```

效果：

保证：

信息守恒。

直观：

普通 residual：

水管：

每层加水

最后爆管

mHC：

每层有阀门

控制信息流

7. 创新三：Muon optimizer

这是训练方面巨大变化。

传统：

AdamW：

维护：

momentum

variance

参数：

$$m_t, v_t$$

Muon：

核心：

对梯度矩阵做：

orthogonalization

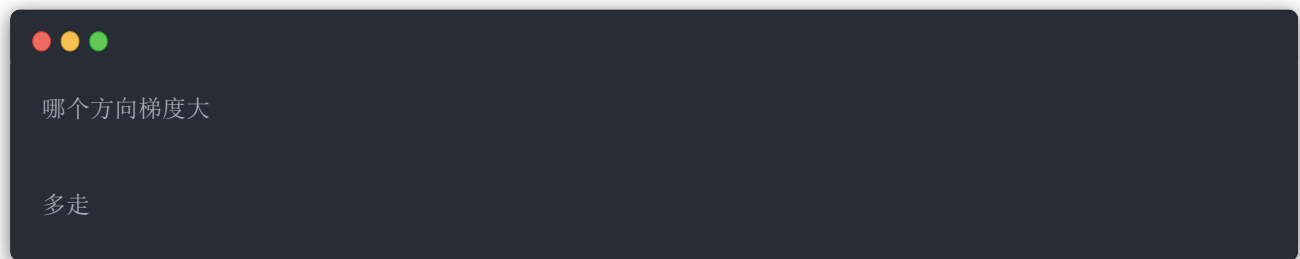
类似：

让更新方向：

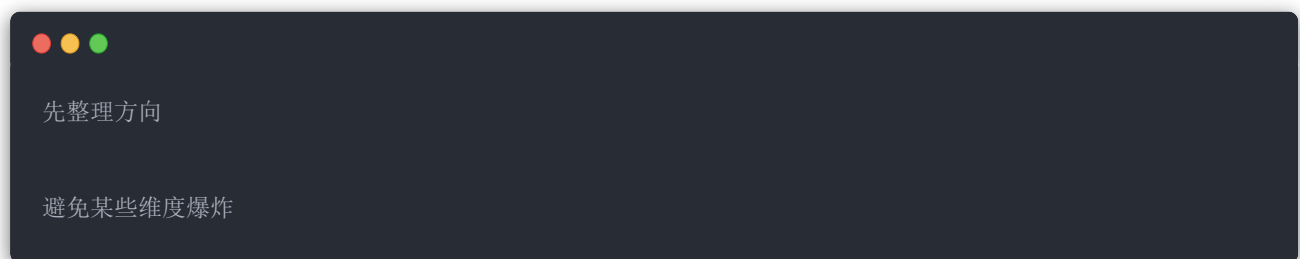
更加均衡。

简单理解：

Adam：



Muon：



对于：

1T+ MoE：

非常重要。

8. 创新四：FP4 Quantization Aware Training

这个和你 B200 FP4 kernel 强相关。

以前：

训练：

BF16/FP8

推理：

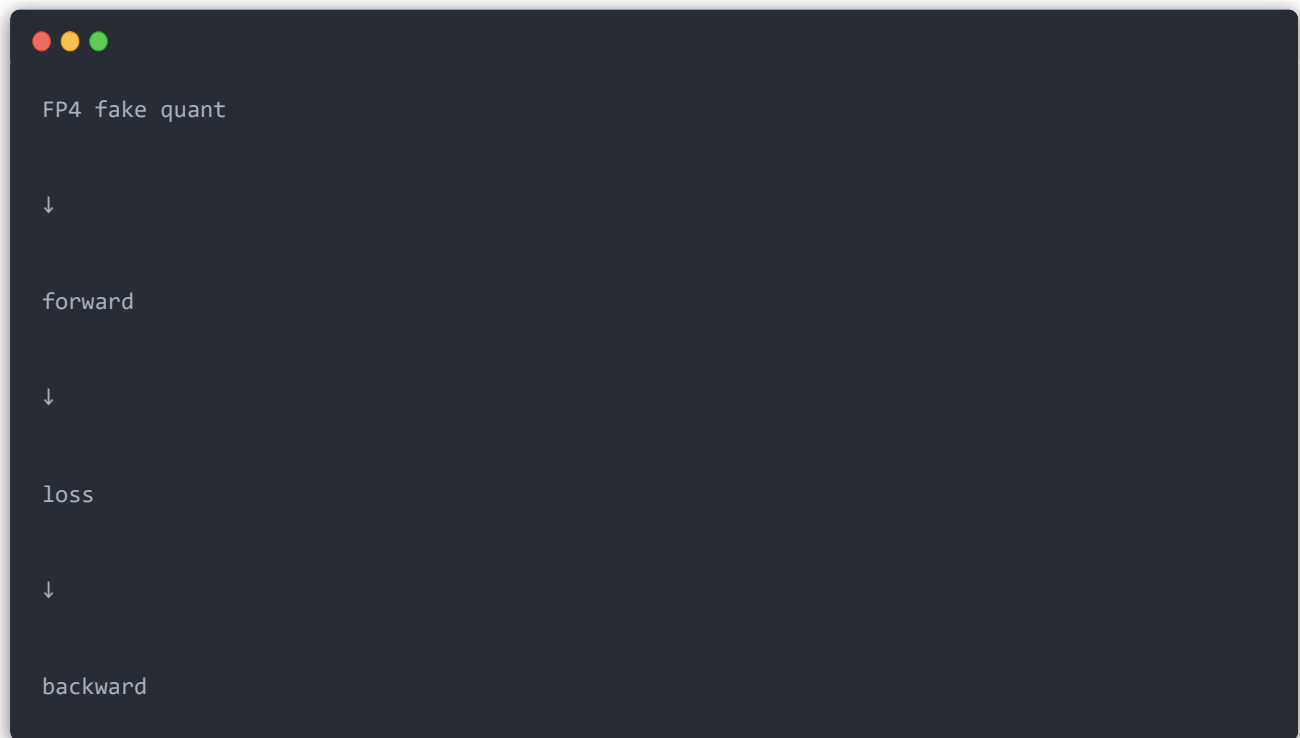
量化。

V4：

训练阶段：

直接考虑 FP4。

流程：

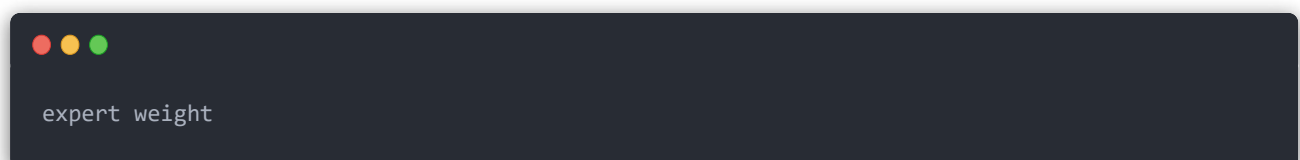


主要用于：

MoE expert weights

为什么？

MoE 最大内存：



不是 attention。

例如：

1.6T 参数：

如果 BF16：

$$1.6T \times 2bytes ===== 3.2TB$$

FP4：

$$1.6T \times 0.5byte ===== 800GB$$

直接降低。

([Reddit][4])

9. MoE 结构

V4 仍然：

Sparse MoE。

例如：

总：



```
384 experts
```

每 token：



```
activate:
```

```
6 experts
```

```
+
```

```
shared expert
```

所以：

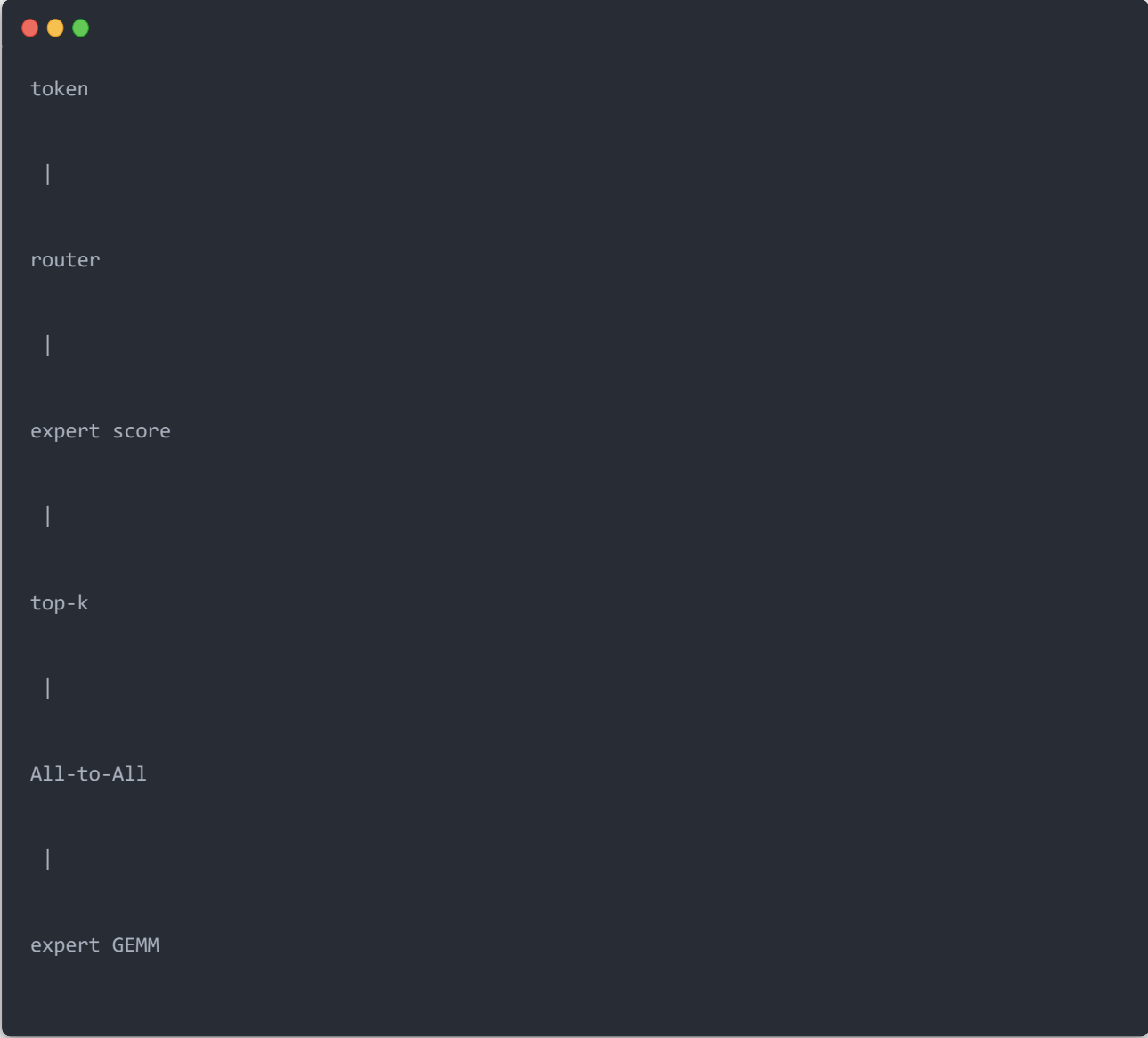
参数：

巨大。

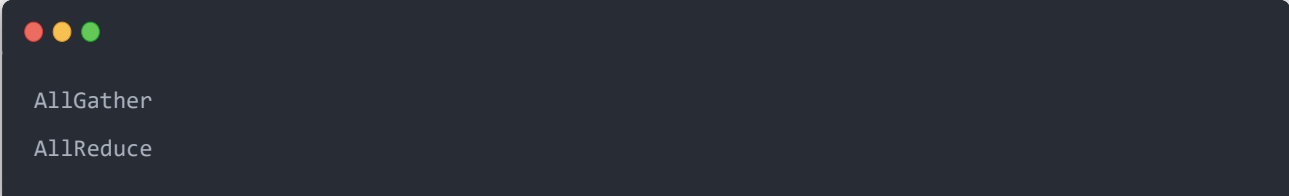
计算：

可控。

流程：



这和你之前 Nsys:



完全对应。

10. 训练流程

DeepSeek V4:

不是简单:

pretrain -> SFT。

大致:



Pretrain

32T+ tokens

|

|

v

Long context training

|

|

v

SFT

|

|

v

Domain RL

|

|

v

官方披露 V4 系列预训练超过 32T 高质量 token，并进行了完整 post-training。([隐敏众][5])

11. RL 部分

延续 DeepSeek-R1：

GRPO。

但是：

V4 更偏：

Agent。

包括：

- coding
- math
- tool use

reward：

不是：



```
answer similarity
```

而是：



```
task success
```

```
+
```

```
execution result
```


类似：

你之前设计 CUDA kernel RL：

reward:

compile
correctness
speedup
anti-cheat

方向非常接近。

12. DeepSeek-V4 vs Kimi K3

你刚刚问 Kimi K3，这两个路线非常有意思：

	DeepSeek-V4	Kimi K3
目标	百万context高效推理	Agent intelligence
Attention	CSA/HCA	KDA
Memory	KV压缩	state memory
Residual	mHC	AttnRes
MoE	Sparse MoE	LatentMoE
训练重点	效率	Agent RL

简单说：

DeepSeek-V4: 优化 Transformer 的信息存储效率

减少KV
减少通信
减少参数成本

Kimi K3: 重新设计信息流

```
token memory
+
depth memory
+
experience memory
```

一句话总结：

DeepSeek-V4 不是简单升级 V3，而是围绕“超长上下文 MoE 推理成本”重新设计 Transformer：用 CSA/HCA 压缩 token memory，用 mHC 稳定 depth memory，用 FP4 降低 parameter memory，用 MoE 保持能力扩展。